

Datatypes

Seppur javascript non è un linguaggio fortemente tipizzato ma debolmente tipizzato e ciò implica che nella sua sintassi non viene specificato il tipo di un dato ma lo si verifica a dato inizializzato è comunque caratterizzato da alcuni tipi.

I tipi di dato in Javascript sono:

Number

String

Boolean

Object

let num = 5; valore di tipo Number fa riferimento a tutti i valori numerici,

let nome = 'fabio'; valore di tipo String e fa riferimento a tutti i caratteri alfanumerici contrassegnati da single quote ' ' o double quote " " ,

let bool = true; valore di tipo Boolean fa riferimento ai valori logici come true o false, let oggetto = {nome:'fabio'}; valore di tipo object e fa riferimento a proprietà e valore di un oggetto.

Typeof

E un comando di javascript utilizzato per verificare in output il tipo di valore assegnato dalla variabile.

Esempio

```
let nome = 'fabio'; Console.log(typeof(nome));
```

Output: String

Array

Gli array sono strutture di dati per gestire liste statiche, lo si può paragonare ad un contenitore che può ospitare n elementi di diverso tipo ed ogni elemento inserito ha un indice o posizione. Ad esempio il primo elemento inserito avrà come posizione il numero 0, nell'esempio sottostante è stato dichiarato un array con quattro valori all'interno dove l'ultimo valore avrà posizione numero 3.

```
let numeri = [1,2,3,4];

let un= numeri.unshift('0'); // inserisce il valore alla prima posizione let pop =
numeri.pop();// rimuove l'ultimo elemento dell'array

let push = numeri.push(5);inserisce un nuovo elemento all'ultima posizione numeri =
numeri.sort();// ordina l'array in modo crescente

console.log(numeri); // ritorna gli elementi dell'array console.log(numeri[1]);//
ritorna l'elemento in seconda posizione console.log(numeri.length); // ritorna la
lunghezza dell'array

numeri.forEach(function(elem,index){

console.log(elem+' '+ index);// itera elemento e indice

})
```

Array multidimensionale

```
Let multi = [[1,'anna'],[2,'paolo'],[3,'chiara']];
```

```
console.log(multi); // le tre dimensioni dell'array con gli elementi inseriti
```

Objects

Gli objects in javascript sono caratterizzati da due entità: proprietà e valore, dichiarate all'interno di una variabile come nell'esempio che segue.

Esempio

```
let corso = {informatica:'javascript'}; informatica =
```

proprietà javascript = valore

Un altro esempio nell'utilizzare un dato diverso può avvenire attraverso una funzione all'interno della mia variabile corso oppure avere altri objects con proprietà e valore.

Esempio

```
let corso = {informatica:'javascript', studenti:5, formazione: function()
```

```
{
```

```
return this.informatica // la keyword this fa riferimento alla variabile corso
```

```
}
```

```
};
```

`console.log(corso.formazione);` in output avremo l'azione inserita all'interno della funzione ovvero di tornare il valore di informatica, oppure di aggiungere una nuova proprietà chiamata già dalla variabile.

Esempio `corso.ore =`

`45;`

`console.log(corso.ore);`

Function e Scope

Una Funzione svolge una particolare azione all'interno di un blocco di codice e in javascript vengono spesso utilizzate.

Una funzione viene definita e successivamente richiamata per svolgere le istruzioni che vengono inserite nel blocco di codice.

Una funzione seguita dalla keyword `function` deve avere un nome e può avere dei parametri passati in ingresso come nell'esempio sottostante `num 1` e `num2`, terminando con il blocco di codice `{}` dove all'interno devono essere presenti le istruzioni che la funzione dovrà eseguire.

Abbiamo il comando `return` per le azioni da svolgere come il prodotto dei parametri inseriti in ingresso.

Al di fuori del blocco di codice della funzione successivamente viene richiamata, inserendo in ingresso il valore dei parametri e il `console.log` per ottenere l'output.

Lo scope di una variabile come `action` viene detta locale se dichiarata all'interno della funzione, tutte le variabili dichiarate al di fuori delle funzioni hanno scope globale e quindi visibili in tutto il nostro file `js`.

Una variabile con scope globale può essere utilizzata all'interno di una funzione, come mostrato nell'esempio e non viceversa.

```
let action = 'function'; // scope globale
```

```
function myfunct(num1,num2){// dichiaro i parametri action =  
'function'; // scope locale  
return num1*num2;  
}  
  
let fun = myfunct(5,7);// parametri inizializzati console.log(fun);
```

Object Math

L'oggetto math mette a disposizione diversi metodi aritmetici che normalmente vengono utilizzati per esigenze specifiche all'interno di un programma.

Vi mostrerò alcuni metodi utilizzati nell'oggetto math.

```
console.log(Math.E)// numero aritmetico di Eulero console.log(Math.PI) numero  
Pigreco aritmetico  
console.log(Math.abs(5))// ritorna il valore assoluto passato come parametro  
console.log(Math.ceil(5.3))// forza il decimale al numero più grande console.log(Math.floor(4.6))//  
forza il decimale al numero più piccolo console.log(Math.sqrt(81)) // radice quadrata del numero  
passato come parametro
```

```
console.log(Math.round(9.7))//se il decimale è minore di 5 console.log(Math.pow(9,2))//  
esponenziale del primo numero passato in ingresso console.log(Math.random()*5)+1); //genera  
numeri casuali da 1 a 5 console.log(Number.MAX_VALUE); //valore massimo assoluto di Number  
console.log(Number.MIN_VALUE);//valore minore assoluto di Number
```

Object Date

Oggetto Date permette di lavorare con le date ovvero metodi per impostare o leggere giorno, mese, anno, ore, minuti e secondi.

Un altro vantaggio è di poter convertire una data in una stringa attraverso il metodo toString.

Esempio

```
let data = new Date
```

```
console.log(data.getFullYear());// output anno corrente  
console.log(data.getDay());// giorno corrente console.log(data.getHours());// ora  
corrente
```

```
let date = new Date(" January 20,2021 11:20:25) ; date.setDate(date.getDate()+5);// imposta una  
data con un counter di +5 day console.log(date); //output January 25, e non più 20 come lo stato  
iniziale
```

String

ToUpperCase e ToLowerCase

Sono metodi(azioni) utilizzati in javascript per la conversione di una stringa in minuscolo.

Esempio “fabio” e la converto in maiuscolo attraverso il metodo ToUpperCase. ToLowerCase viene utilizzato per la procedura inversa da stringa compilata in maiuscolo viene convertita in minuscolo.

Esempio

ToUpperCase

```
Let nome = 'fabio';
```

```
Console.log(nome.ToUpperCase);
```

ToUpperCase

```
Let nome = 'FABIO';
```

```
Console.log(nome.ToLowerCase);
```

* il . chiamato dot notation viene inserito per poter utilizzare o chiamare il metodo definito.

ParseInt e toString

Sono metodi chiamati da una variabile convertendone il suo tipo di dato.

ParseInt viene utilizzato quando ho una stringa con valore numerico ed ho bisogno che sia di tipo Number e non String, viceversa il toString converte un valore di tipo Number o Date(riferito alle date) in un tipo di dato String.

Esempio

parseInt

```
let val = '2020';  
Console.log(parseInt(val));  
Output: 2020
```

toString

```
let val = 100;  
Console.log(val.toString());  
Output: "100"
```

Handling Exception

Durante lo sviluppo di un programma può succedere di prevenire la possibilità che si verifichino anomalie del programma e in Javascript viene eseguito con dei comandi specifici: try-catch-finally all'interno di blocchi di codice.

Esempio

```
try {  
    let exc = 5/0; // nel try si inserisce l'operazione da svolgere  
    } catch (error) { // il comando catch all'interno ha il parametro error e viene inserito in caso di  
        un'anomalia del programma e non causarne il crash  
    }  
    finally{ // le istruzioni inserite nel comando finally verranno comunque eseguite sia che si  
        presenti o meno l'anomalia  
        console.log('Eccezione Gestita')  
    }  
}
```


Esempio

```
try {  
    let und;  
  
} catch (error) {  
  
}  
  
finally{  
  
    console.log('Eccezione Gestita');  
}
```

Esempio

```
function getRectArea(width, height) {  
    if (width<height) { // comando throw permette di personalizzare l'eccezione e di farla  
        eseguire se si verifica tale condizione  
        throw 'Failed';  
    }  
}
```

```
try { getRectArea(3,5);  
    } catch (error) { // eccezione gestita console.log(error);  
  
    }
```

Debugging Code

Il debug code solitamente viene utilizzato in fase di test prima che il programma vada in produzione per verificare se il codice sviluppato esegue tutto correttamente fissando dei singoli punti di test del codice chiamati breakpoint.

Una volta fissati i punti di partenza e arrivo del debug del codice è possibile procedere utilizzando i comandi Step into e Step over, Step over esegue ogni singola riga di codice del programma, mentre Step into va più nel dettaglio del parametro o del metodo.

Esempio Step over F10 Step into F11 Start Debug F5 Let

```
num1=50; num2=
```

```
= 25;
```

```
if (num1>num2){

    console.log(true);
}

else {

    console.log(false);
}

if (num2<num1){

    console.log('RIPROVA');
}

    • debugger;
```

In questo esempio ho inserito due breakpoints: il primo alla prima riga il secondo inserendo alla riga dove è presente la keyword debugger, premendo F5 do inizio al debug e successivamente deciderò se procedere con Step over F10 procedendo in sequenza del codice oppure se desidero andare nel dettaglio dell'istruzione o del comando ad esempio dove è presente console.log utilizzerò Ste pinto F11. Nell'output della console del mio editor di sviluppo potrò verificare l'analisi del mio debug.

Use strict

Use strict annota ad i browser più recenti la modalità di scripting è stata definita “modalita rigorosa” è stata introdotta in EcmaScript 5 permettendo al programmatore di scrivere un buon codice, ad esempio non permette di utilizzare una variabile non inizializzata.

Esempio

```
"use strict"; let
num1 = 50; let
num2
= 25;

let cod; // errore non consentito con use strict

if (num1>num2){

    console.log(true);
}

else {

    console.log(false);
}

if (num2<num1){

    console.log('RIPROVA');
}
```

Introduzione alla Programmazione ad Oggetti

Concetto di classe:

La classe in informatica è un modello così come nella vita quotidiana può essere una persona o un bene materiale come l'automobile.

Una classe ha delle proprietà, dei metodi e tante altre caratteristiche come visto in precedenza. Gli aspetti principali della classe sono i metodi i quali svolgono determinate azioni all'interno della classe e dalle variabili di istanza che permettono alla classe di essere istanziata ed invocare proprietà o metodi.

```
class Studente { // nome della classe
  nome: string // proprietà di classe
  cognome:string
  constructor(nome:string,cognome:string) { //il costruttore referencia e inizializza le proprietà della classe
    this.nome = nome
    this.cognome = cognome
  }
  getStudente(): string { //metodo di classe
    return this.nome
  }
}
let studente = newStudente('mario',rossi'); // la classe viene istanziata e viene creata la variabile di istanza

let stud = studente.getStudente
// istanza studente invoca il metodo
```

Metodi static e metodi di istanza

I metodi static hanno caratteristiche differenti rispetto ai classici metodi di istanza tra cui:

Possono essere invocati solo dalla classe

Possono collaborare solo con caratteristiche definite static.

Differenza tra metodi e funzioni

Seppur il loro scopo è di svolgere azioni ma hanno collocazioni differenti.

Una funzione può esistere anche senza una classe.

Una funzione non ha proprietà di lettura o scrittura come accade nei metodi.

I metodi sono una caratteristica della classe.

Nei metodi vige la proprietà getter and setter per la lettura e scrittura del metodo.

Un metodo viene invocato dalla classe o dall'istanza mentre una funzione dispone di una chiamata.

I paradigmi dell'OOP

Incapsulamento-Ereditarietà-Polimorfismo

Incapsulamento

Il concetto dell'incapsulamento è quello di mantenere riservate le informazioni solo all'interno della classe che li definisce attraverso il modificatore di accesso private,

Tutte le proprietà o metodi con modificatore private non potranno avere accesso diretto(istanza) da una classe esterna.

Ereditarietà

E' possibile ereditare tutte le caratteristiche (proprietà o metodi definiti public) appartenenti ad una data classe X utilizzando la keyword extends su di una nuova classe chiamata Y.

La nuova classe Y prenderà il nome di sottoclasse e la classe X prenderà il nome di superclasse.

Polimorfismo

Si intende 'polimorfico' la ridefinizione di un metodo appartenente alla superclasse X mediante l'utilizzo dell'ereditarietà nella sottoclasse Y